

Course 6: Adapting a Deep Learning model



IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Last session

- 1 Foundation models
- 2 Multimodal Foundation models

Today's session

- Transfer Learning
- Fine-tuning
- PEFT

Adapting a model?

- A model (e.g., a foundation model) can extract general-purpose representations and understand your data at a high level (see previous labs).
- But it may not be suited for **your particular set-up** (e.g., medical imaging, which requires a precise understanding of MRI and scanners).

Definition

“Adapting” a model means tailoring it to one’s specific needs!

Why adaptation?

It saves an enormous amount of data, computation, and time. You are “transferring” the knowledge from the general task to your specific one.

Adaptation process

- 1 Pre-training (Done by others):** A giant model (e.g., GPT-3, ViT) is trained on a massive, general dataset (e.g., the whole internet, ImageNet).
→ This model has learned how to process this type of data, in a general way.
- 2 Adaptation (Done by you):** You take this powerful, pre-trained model and *adapt* it for your specific, niche task (e.g., classifying medical images).
→ You use a particular set of data and specialize your model on it.

Adaptation techniques

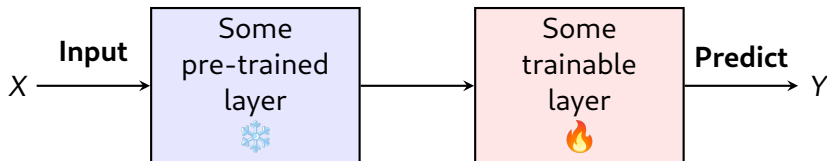
- Transfer Learning
- Fine-tuning
- Parameter-Efficient fine-tuning (PEFT)

Trainable and non-trainable parameters

In adaptation, the model parameters (weights) may be trained or kept fixed ("frozen", *i.e.* they retain the values from the original model).

- Trained parameters are denoted with a 🔥 symbol;
- Frozen (fixed) parameters are denoted with a ❄️ symbol.

Example of a model representation:



Outline

1 Transfer Learning

2 Fine-Tuning

3 PEFT

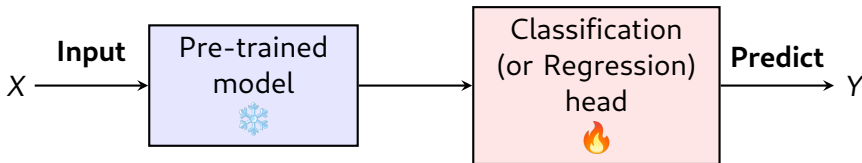
4 In practice

Transfer Learning (1/2)

Tentative definition

Transfer learning consists of using the pre-trained model for feature representation, and adding one (or several) classification layers on top of the representation.

This is also called “Linear probing” when the classification head is linear.



Note: this is what we did by using foundation model embeddings in previous labs!

Advantages

- Easy
- Pre-trained model can be used off-the-shelf
- Training cost depends on the additional layers only (can be lightweight)
- Can work with small datasets
- Only require inference from the model $\Rightarrow$ can work with a "black-box" closed model

Disadvantages

- Heavily depends on the pre-training
- Representations may not be relevant or expressive enough
- May not benefit ("scale") from large datasets

Outline

1 Transfer Learning

2 Fine-Tuning

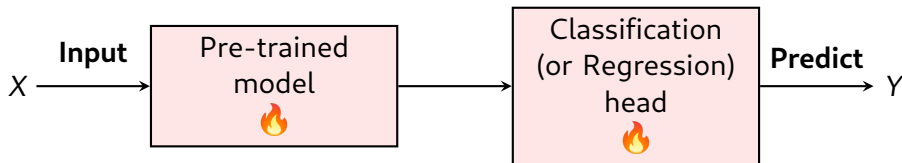
3 PEFT

4 In practice

Fine-tuning (1/2)

Tentative definition

Fine-tuning consists of re-training the whole model, with or without additional layers.
The training is "fine-grained," and lighter than the initial pre-training.



Advantages

- Very efficient
- Generally obtains the best performances
- Benefits from large datasets ("scale")

Disadvantages

- Depending on the size of the model, the training can be computationally expensive (or even prohibitive)
- Fine-tuning can destroy the pre-training ("catastrophic forgetting")
- Requires a large dataset
- Requires access to the model (open-source)

Outline

1 Transfer Learning

2 Fine-Tuning

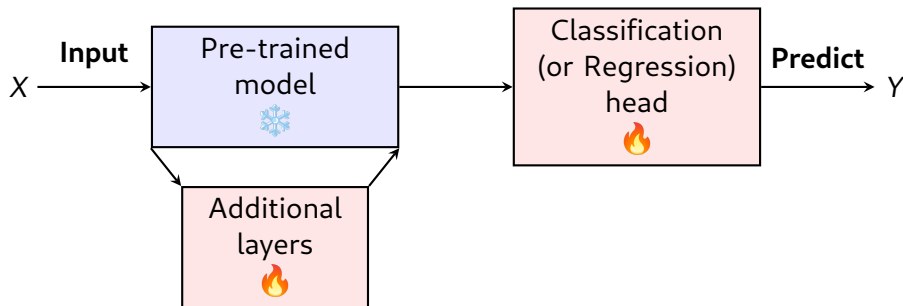
3 PEFT

4 In practice

Parameter-efficient fine-tuning (1/2)

Tentative definition

Performs fine-tuning by only training a limited set of parameters.



Parameter-efficient fine-tuning (2/2)

Advantages

- Efficient
- Lightweight
- Can scale with both small and large datasets

Disadvantages

- Needs some parametrization (hyperparameters, architecture choices, etc)
- Requires access to the model (open-source)
- May only lightly adapt a model, which may not be sufficient in some applications.

LoRA

Low-Rank Adaptation (LoRA) is one of the most standard PEFT methods.

- Lightweight, and super efficient
- Easy to define with HuggingFace (see the doc by clicking here)

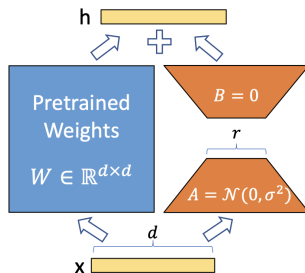


Figure 1: Our reparametrization. We only train A and B .

LoRA: <https://arxiv.org/pdf/2106.09685v1/1000>

LoRA, in detail

- LoRA was developed for attention layers (but it may also be used for other types of layers, on particular MLP).
- LoRA consists of two low-rank matrices $A \in \mathbb{R}^{h \times r}$ and $B \in \mathbb{R}^{r \times h}$ (parameter r must be set by the user)

The LoRA equation (for layer l):

Pre-trained weights  LoRA weights 

$$x^{(l+1)} = W_0 x^{(l)} + AB x^{(l)}$$

LoRA largely reduces the number of trainable parameters! ex: ViT

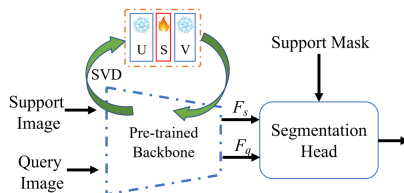
- Full fine-tuning: 86M
- LoRA ($r = 4$): 221k (0.258%)

Singular Value Fine-tuning (SVF)

SVF

Singular Value Fine-tuning (SVF) consists of fine-tuning the singular values and keeping the rest frozen

- Very lightweight (in general less than 1% of the parameters are fine-tuned)
- Can provide some interpretability



SVF: <https://arxiv.org/pdf/2206.06122>

BitFit

BitFit freezes all the weights and only fine-tunes the biases or a subset of the biases

- Very few parameters to fine-tune (sometimes less than 0.1%)
- can be deployed into multiple tasks easily

$$\mathbf{Q}^{m,\ell}(\mathbf{x}) = \mathbf{W}_q^{m,\ell} \mathbf{x} + \mathbf{b}_q^{m,\ell}$$

$$\mathbf{K}^{m,\ell}(\mathbf{x}) = \mathbf{W}_k^{m,\ell} \mathbf{x} + \mathbf{b}_k^{m,\ell}$$

$$\mathbf{V}^{m,\ell}(\mathbf{x}) = \mathbf{W}_v^{m,\ell} \mathbf{x} + \mathbf{b}_v^{m,\ell}$$

BitFit: <https://arxiv.org/pdf/2106.10199>

Outline

1 Transfer Learning

2 Fine-Tuning

3 PEFT

4 In practice

Adapting a model, in practice

See the tutorial on the labs page to adapt your own model!

mee-labs.gitlab-pages.imt-atlantique.fr/intro2ai/adapt